INICIANDONOS CON ARDUINO

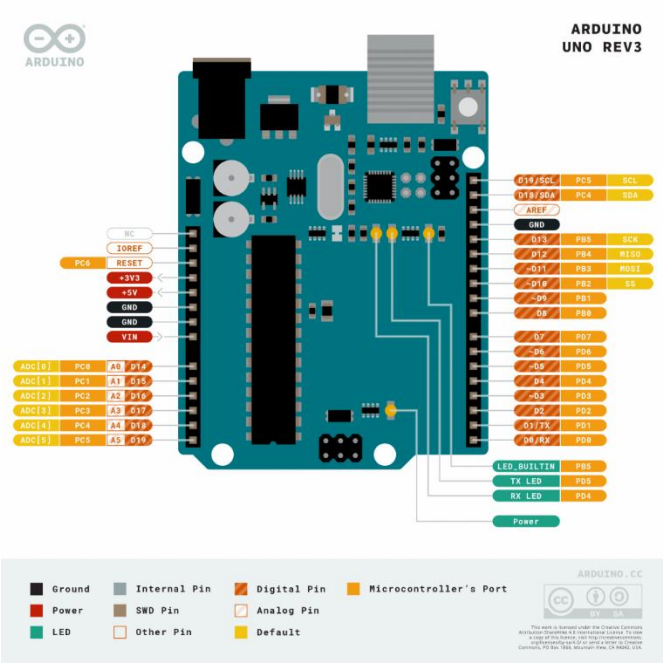
TECNOLOGÍA Y DIGITALIZACIÓN 3º ESO

1. ¿Qué es arduino?

Arduino es una placa electrónica programable.

Esto significa que podemos escribir instrucciones (un programa) para que haga cosas automáticamente según como lo programemos.

Vamos a ver las conexiones básicas que debemos conocer de un Arduino:



Pines de alimentación

Reset – Mediante este pin se puede resetear o reiniciar el microcontrolador. Se conecta a tierra para reiniciar (activo a nivel bajo) e internamente en la tarjeta este pin tiene una resistencia pull-up.

+3.3V – Este pin esta conectado a la salida del regulador de 3.3 volts de la tarjeta. Normalmente solo debe ocuparse como una salida de voltaje que puede servirnos para alimentar circuitos de 3.3 volts de bajo consumo.

5V – Este pin es la salida del regulador de 5 volts de la placa o bien el voltaje que proviene del conector USB dependiendo de donde estemos alimentando la placa.

GND – Estos pines son la tierra o masa del circuito, está conectado al negativo de la alimentación.

VIN – Voltaje de entrada sin regular, funciona como entrada / salida de voltaje de alimentación de 9 volts.

Pines analógicos (ADC)

Los pines analógicos permiten leer señales analógicas como la voz, la cantidad de luz o la temperatura. Esto se logra mediante un circuito especializado dentro del microcontrolador conocido como convertidor analógico a digital (ADC o Analog to Digital Converter).

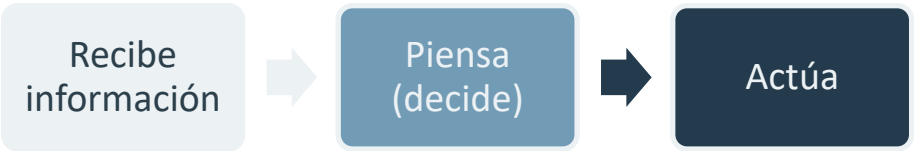
Este circuito permite convertir un voltaje presente en los pines analógicos A0 al A6 en un número que va del 0 a 1023. Al representar la magnitud analógica como un número, el microcontrolador puede entenderlo y usarlo para realizar cálculos o tomar decisiones.

Pines digitales

Los pines digitales se pueden utilizar para conectar dispositivos como LEDs, relevadores, drivers de motores, botones, switches magnéticos (reed switch) etc. Los pines digitales pueden configurarse como entradas o salidas, dependiendo del tipo de sensor o dispositivo que deseamos conectar.

¿Cómo funciona Arduino?

Arduino trabaja como un sistema muy sencillo:



2. ¿Qué es programar?

Programar es decirle paso a paso a una máquina lo que tiene que hacer. Para decírselo hay que utilizar un lenguaje de programación que sea capaz de compilar. Arduino utiliza C++ como lenguaje de programación.

Muy importante Arduino **NO piensa por sí solo**, solo sigue órdenes. Por lo que, si el programa está mal, el funcionamiento no será correcto.

3. Estructura de un programa

Funciones esenciales en Arduino

Todo programa de Arduino tiene esta forma:

```
void setup() {

}

void loop() {

}
```

¿Qué significa esto?

`setup()` es un comando que solo ejecutamos una vez en todo el programa y se utiliza para preparar el circuito, es decir, indicar los pines que vamos a utilizar, que conexiones vamos a necesitar,…

`loop()` es un comando que se ejecuta en bucle sin parar

Y recordemos que las llaves { } definen bloques de código, es decir, agrupan instrucciones que pertenecen a una misma función o estructura de control. Y siempre debe haber una llave de apertura y una llave de cierre.

A continuación, un ejemplo, dónde dentro de setup () indico que voy a utilizar el pin 13 como salida, y en loop () indico que el pin 13 va a estar encendido. Ahora lo importante es centrarse en los comentarios marcados con // al inicio.

```
void setup() {
  pinMode(13, OUTPUT); //Configuro el pin 13 como una salida
}

void loop() {
  digitalWrite(13, HIGH); // Indico que le pin 13 siempre va estar
encendido
}
```

Variables

Otro concepto que debemos tener clarísimo a la hora de diseñar un programa son las variables.

Una variable es una “caja” donde guardamos información, para poderla utilizar posteriormente y tomar decisiones en base a su información almacenada.

Existen variables de varios tipos en función de la información que queramos guardar en la misma, pero los diferentes tipos de variables lo veremos en el siguiente tema. Ahora solo vamos a ver como declarar las variables más sencillas, las variables que se encargan de almacenar un número entero.

Para crear una variable basta con utilizar el comando `int` seguido por el nombre que queramos asignarle a la variable e indicar su valor.

A continuación, tenemos un ejemplo de cómo crearíamos una variable llamada edad, en la cual queremos guardar que la edad es de 30 años.

```
int edad = 30;
```

Pines de arduino

Arduino tiene "patas" (pines) para conectar componentes, dispone de dos tipos de pines digitales y analógicos como hemos visto en el primer punto.

Pines digitales: Se pueden configurar como entrada o salida y pueden estar en dos estados.

HIGH → encendido

LOW → apagado

Pines analógicos: Sirven para leer valores variables, es decir, solo se pueden utilizar como entrada.

Cómo utilizar un pin como salida

Para configurar un pin utilizo el comando `pinMode()` dónde como parámetros le tengo que indicar el número de pin que voy a utilizar y si va a ser una input o una output.

A continuación, un ejemplo del comando que utilizaría para indicarle a Arduino que el pin 13 lo voy a utilizar para enviar una señal. Este comando como es de preparación del circuito debería ir dentro de `setup ()` como hemos visto con anterioridad.

```
pinMode(13, OUTPUT);
```

Y ahora supongamos que al output 13 he conectado un led que quiero encender, para ello necesitamos utilizar el comando `digitalWrite ()` dónde como parámetros le indicamos el número de pin sobre el que queremos actuar y el estado que queremos poner, en este caso encendido (High).

```
digitalWrite(13, HIGH);
```

Este comando es para ejecutarse por lo que debe ir definido dentro de `loop ()`.

Y si en caso contrario, quisiéramos apagar el LED lo que haríamos sería lo mismo pero indicándole como parámetro LOW.

```
digitalWrite(13, LOW);
```

Cómo utilizar un pin como entrada

A continuación, vamos a ver cómo podemos configurar el pin A0 para leer a información que nos ofrece un sensor. Al tratarse de un sensor la información que nos dará será un número

variable en el tiempo con un rango amplio de valores y vamos a suponer que siempre nos da un número entero.

Para leer la información de un pin analógico se utiliza el comando `analogRead ()`, dónde le insertamos como parámetro el pin desde el cual queremos leer. Pero claro la información que leemos debemos almacenarla en algún sitio para poder utilizarla, por lo que necesitamos utilizar las variables que hemos visto con anterioridad.

Por ejemplo, podemos crear una variable llamada luz que sea de tipo entero (porque anteriormente hemos dicho que suponemos que le sensor nos ofrece valores enteros) para almacenar la información obtenida del sensor.

Por tanto, el comando para leer la luz de un sensor conectado al pin A0 quedaría de la siguiente manera:

```
int luz = analogRead(A0);
```

Hasta ahora hemos visto como configurar un pin, como declarar una variable y como almacenar un valor leído de un pin.

Tomar deciciones (IF)

¿Qué es un if?

Un `if` significa: “SI pasa algo, haz esto”. Es una estructura condicional cuya función principal es tomar decisiones automáticas basándose en una condición. Si la condición es verdadera (Se cumple) ejecuta la orden que le hemos indicado, en caso contrario (la condición no se cumple) no hace nada.

Por ejemplo, si seguimos con los ejemplos anteriores podemos decir, si está oscuro enciende la luz.

Recordemos que un sensor mide luxes para saber la luz que hay en un área, y podemos considerar si hay menos de 300 luxes en un área, está esta oscura.

Entonces lo que tengo que hacer es indicar que si la variable luz creada anteriormente tiene un valor inferior a 300 me encienda la luz.

```
if (luz < 300) {
  digitalWrite(13, HIGH);
}
```

IF + ELSE

La estructura de un `if` se puede completar con un `else`. De forma que “Si pasa algo, haz esto sino haz esto otro”. Es una estructura condicional cuya función principal es tomar decisiones automáticas basándose en una condición al igual que el `if` pero le añadimos una orden a ejecutar en caso de que la condición no se cumpla. Si la condición es verdadera (Se cumple) ejecuta la orden que le hemos indicado, en caso contrario (la condición no se cumple) ejecuta la orden que le hemos indicado en el else.

A continuación, seguimos con el ejemplo de antes:

Si está oscuro (menos de 300 luxes) enciende la luz, pero sino apaga (entiendo que hay suficiente luz y así podemos ahorrar energía)

```
if (luz < 300) {
  digitalWrite(13, HIGH);
} else {
  digitalWrite(13, LOW);
}
```

A continuación, el programa completo que hemos ido realizando mediante los ejemplos:

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  int luz = analogRead(A0);

  if (luz < 300) {
    digitalWrite(13, HIGH);
  } else {
    digitalWrite(13, LOW);
  }
}
```

Consejo: para que le programa sea más entendible podemos crear variables con un nombre identificativo que almacenen el número del pin que queremos leer/escribir, de esta forma el programa queda más claro para su interpretación.

Por ejemplo, podemos nombrar que el pin 13 se llame led ya que es lo que tendremos conectado, y que el pin A0 dónde tenemos el sensor conectado se llame sensor.

```
int led = 13;
int sensor = A0;
```

De esta forma el código final quedaría de la siguiente manera:

```
int led = 13;
int sensor = A0;

void setup() {
  pinMode(led, OUTPUT);
}

void loop() {
  int luz = analogRead(sensor);

  if (luz < 300) {
    digitalWrite(led, HIGH);
  } else {
    digitalWrite(led, LOW);
  }
}
```